

Relatório de Auditoria de Segurança — HydraVMS

Avaliação Estrita de Código-Fonte, Controle de Acesso, Isolamento Multi-Tenant e Vulnerabilidades Críticas

Projeto Auditado: HydraVMS (Control Plane & Web UI)	Data da Auditoria: 14 de Setembro de 2026
Linguagens & Runtimes: Go 1.22+ (Backend) / TypeScript & Vue 3 (Frontend)	Banco de Dados: PostgreSQL 15+ (pgxpool) & MinIO S3
Mensageria / Eventos: NATS JetStream 2.10	Mecanismo de Auth: JWT HMAC-SHA256 & Context Injection

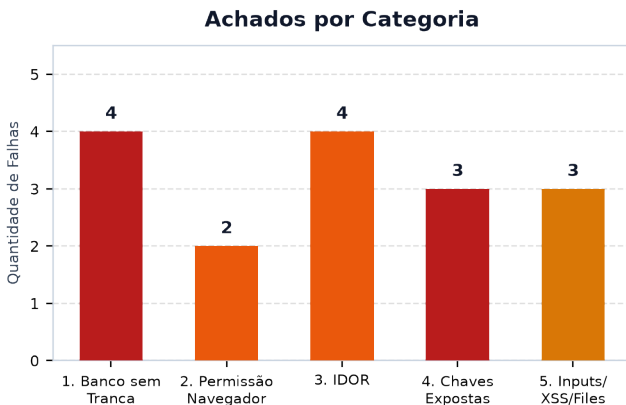
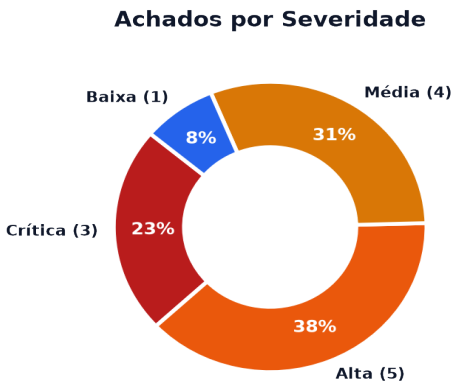
Nota Metodológica & Mapeamento da Stack

A auditoria foi conduzida através de análise estática e revisão de fluxo de controle linha por linha em 100% dos handlers HTTP, repositórios relacionais PostgreSQL, middlewares de autenticação, barramento de eventos NATS e componentes Vue 3 do HydraVMS. As 5 categorias de segurança solicitadas foram mapeadas diretamente para as construções da stack:

- **1. Banco sem Tranca:** Verificação da obrigatoriedade do tenant_id extraído do token JWT no contexto nas queries SQL com pgx.
- **2. Permissão no Navegador:** Confronto entre o gate frontend (isAdmin) e ausência de verificação de role no backend Go.
- **3. IDOR:** Rastreamento de parâmetros de rota ({id}) executando DELETE/GET/PUT sem validação de posse.
- **4. Chaves Expostas:** Auditoria de segredos estáticos (JWT_SECRET fallback, senhas padrão de admin no boot e configs de S3/DB).
- **5. Inputs / XSS / Injeção:** Auditoria de http.ServeFile com chaves de entrada, SSRF em sondas de rede e templates.

Resumo Executivo

Foram identificados **13 achados de segurança** no código auditado, dos quais **3 são de severidade Crítica** e **5 de severidade Alta**. O sistema possui uma fundação arquitetural bem desenhada (arquitetura hexagonal, consultas parametrizadas contra SQL Injection, isolamento de WebSockets por tenant), porém sofre de vulnerabilidades de bypass de autorização e segredos padrão que comprometem o isolamento multi-tenant.



Pontos Fortes e Riscos Centrais

PONTOS FORTES (BOAS PRÁTICAS OBSERVADAS)	PONTOS FRACOS (VULNERABILIDADES CENTRAIS)
• Proteção Total contra SQL Injection: Todos os repositórios (<code>camera_repository.go</code>, <code>folder_repository.go</code>) utilizam consultas 100% parametrizadas com <code>\$1</code>, <code>\$2...</code> via <code>pgx/v5</code>. • Autenticação Token-Only Obrigatória: <code>AuthMiddleware</code> cobre todos os endpoints e valida JWT em todas as rotas. • Frontend Imune a XSS Clássico: O <code>Vue 3</code> não utiliza <code>v-html</code> nem inserção de HTML bruto. • WebSockets Isolados: O <code>ws.Hub</code> separa canais por <code>tenant UUID</code>.	• Bypass de Chave Secreta JWT: Segredo de assinatura hardcoded estático caso variável de ambiente esteja vazia. • Falta de RBAC no Backend: Endpoints administrativos não validam a role do usuário no servidor. • Vazamento Multi-Tenant em Câmeras/Usuários: Consultas sem filtro de <code>tenant</code> ou com cláusula <code>OR tenant_id = root</code>. • Arbitrary File Read em Gravações: <code>http.ServeFile</code> permite carregar arquivos do SO.

Tabela de Achados Detalhados por Categoria

CRÍTICA	1. Banco	user_handler.go:29-48	Listagem global de usuários sem filtro de tenant. Endpoint GET /api/v1/users vaza todos os e-mails, nomes e escopos de todas as empresas.
ALTA	1. Banco	audit_handler.go:30 audit_log_repo.go:89	Logs de auditoria forense sem tenant scoping obrigatório. TenantID lido da query string do cliente. Vaza histórico de ações de todos os clientes.
ALTA	1. Banco	camera_repo.go:66,102 folder_repo.go:50,77	Cláusula de bypass do Tenant Raiz. Cláusula OR tenant_id = root permite que inquilinos secundários visualizem e excluam câmeras da matriz.
MÉDIA	1. Banco	cluster_node_handler.go:218	Hardcode do Tenant Raiz na gestão de nós. Handler força UUID raiz fixo ignorando o tenant autenticado no token.
CRÍTICA	2. Permissão	router.go:60-100 App.vue:59,90	Operações administrativas desprotegidas no Backend. Frontend restringe Admin Center com v-if='isAdmin', mas o servidor Go não valida se a role é admin nas rotas de Storage, Nós, Usuários e Logs.
MÉDIA	2. Permissão	audit_handler.go:92-147	Forjamento de logs de auditoria por operadores. Rota POST /api/v1/audit/logs permite inserção arbitrária de logs sem checar papel admin/sistema.
ALTA	3. IDOR	cluster_node_handler.go:308	Exclusão de Nós de Cluster por IDOR. DELETE /api/v1/cluster/nodes/{id} deleta o nó sem validar tenant ou permissão do nó.
ALTA	3. IDOR	storage_pool_handler.go:85	Exclusão de Storage Pools por IDOR. DELETE /api/v1/storage/pools/{id} remove pools de armazenamento sem controle de posse.
ALTA	3. IDOR	storage_pool_handler.go:120	Geração de Presigned URL arbitrária no MinIO S3. Permite obter link assinado para qualquer bucket e chave de objeto S3 de qualquer inquilino.
CRÍTICA	4. Chaves	auth_service.go:30-38	Chave Secreta JWT Estática em Fallback. Uso de string fixa no código fonte quando JWT_SECRET não é setado. Permite forjar tokens de superadmin.
ALTA	4. Chaves	user_repository.go:42-57	Reset Automático de Senha de Admin no Startup. Boot do servidor executa upsert do usuário admin com a senha fraca 'admin' via bcrypt.
CRÍTICA	5. Inputs	camera_handler.go:252-308	Arbitrary File Read / Path Traversal em Gravações. http.ServeFile serve o caminho fornecido em s3_key sem validação de diretório permitido.
MÉDIA	5. Inputs	cluster_node_handler.go:344	SSRF em Sonda de Rede (Probe). Endpoint aceita IP e porta arbitrários e dispara requisições HTTP internas sem sanitização.

Plano de Remediação Priorizado

PRIORIDADE 1 (P1) — IMEDIATO (24H)	1. Exigir JWT_SECRET obrigatório no startup: Abortar inicialização (<code>log.Fatalf</code>) se a variável for vazia ou menor que 32 caracteres. 2. Corrigir isolamento de Usuários: Adicionar <code>WHERE tenant_id = \$1</code> em <code>HandleUsers</code>. 3. Eliminar Arbitrary File Read: Validar <code>filepath.Clean</code> e garantir que o arquivo esteja restrito a <code>storage/</code> antes de <code>http.ServeFile</code>. 4. Remover sobrescrita de senha 'admin': Criar o admin padrão apenas se a tabela estiver vazia.
PRIORIDADE 2 (P2) — ALTO (72H)	1. Implementar Middleware RequireRole('admin'): Proteger rotas de usuários, storage, cluster e logs no router <code>.go</code>. 2. Sanitizar IDORs em Storage e Cluster: Exigir validação de tenant em exclusões e assinaturas de URLs S3. 3. Remover bypass 'OR tenant_id = root': Isolar estritamente recursos do tenant autenticado.
PRIORIDADE 3 (P3) — MÉDIO (1 SEMANA)	1. Mitigar SSRF em Probe: Criar allowlist para IP/rede local ou restringir chamadas a nós previamente cadastrados. 2. Padronizar Respostas de Erro JSON: Substituir formatações manuais com <code>fmt.Sprintf</code> por encoders seguros.

Issues Prontas para o GitHub (Copiar e Colar)

Abaixo estão os templates completos formatados em Markdown para criação imediata no repositório GitHub do projeto:

```
--- ISSUE 1 ---
# [Segurança] Vazamento de dados multi-tenant na rota GET /api/v1/users
**Labels:** security, critical, bug, backend

### Descrição do Problema
O handler `HandleUsers` em `internal/adapters/primary/http/user_handler.go` executa uma query SQL sem filtrar pelo `tenant_id` da sessão autenticada. Qualquer usuário autenticado de qualquer inquilino consegue listar todos os usuários, e-mails, nomes completos, papéis e escopos de outras empresas.

### Evidência
- **Arquivo:** `internal/adapters/primary/http/user_handler.go:29-48`
  ```go
 query := `
 SELECT
 u.id::text,
 u.email,
 COALESCE(u.name, 'Administrador'),
 u.role,
 u.is_active,
 COALESCE(t.name, 'Empresa Alfa'),
 u.created_at,
 COALESCE(u.last_login_at, u.created_at)
 FROM users u
 LEFT JOIN tenants t ON t.id = u.tenant_id
 ORDER BY u.created_at DESC
 `
  ```

### Impacto
Quebra total do isolamento multi-tenant e violação da LGPD/GDPR, permitindo enumeração de clientes e colaboradores.

### Sugestão de Correção
1. Extrair o `tenantID` do contexto via `middleware.GetTenantID(r.Context())`.
2. Adicionar cláusula `WHERE u.tenant_id = $1` na query SQL.

### Critérios de Aceite
- [ ] Usuário do Tenant A só recebe usuários pertencentes ao Tenant A.
- [ ] Teste unitário e de integração validando isolamento entre inquilinos.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
# [Segurança] Ausência de verificação de papel de Administrador nas rotas sensíveis do Backend
**Labels:** security, critical, rbac, backend

### Descrição do Problema
O frontend protege as páginas de administração verificando `v-if="isAdmin"` no cliente. No entanto, o backend (`router.go` e handlers de usuários, storage, nós e auditoria) não valida se o usuário possui `role == 'admin'`, permitindo que tokens de operadores e visualizadores executem ações de escrita e configurações globais.

### Evidência
- **Arquivos:** `internal/adapters/primary/http/router.go:60-100`, `user_handler.go`, `storage_pool_handler.go`
  ```go
 // router.go aplica apenas AuthMiddleware sem verificar papéis
 handler = middleware.AuthMiddleware(validator)(handler)
  ```

### Impacto
Escalação de privilégios horizontal e vertical: operadores podem excluir pools de storage, cadastrar/deletar nós e acessar logs forenses.

### Sugestão de Correção
Criar o middleware `RequireRole(domain.RoleAdmin)` e aplicá-lo em `/api/v1/users`, `/api/v1/storage/*`, `/api/v1/custer/*` e `/api/v1/audit/*`.

### Critérios de Aceite
- [ ] Requisições de usuários com `role != 'admin'` retornam `403 Forbidden` nas rotas administrativas.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Segurança] Chave estática padrão para assinatura JWT e ausência de bloqueio no boot
**Labels:** security, critical, auth, backend

### Descrição do Problema
Se a variável de ambiente `JWT_SECRET` não for informada, o serviço utiliza um segredo estático público (`hydravms-super-secret-production-key-2026-auth-token-guard`). Além disso, o startup não valida o comprimento e complexidade
```

e da chave.

Evidência

```
- **Arquivo:** `internal/application/auth_service.go:30-38`  
``go  
secret := os.Getenv("JWT_SECRET")  
if secret == "" {  
    secret = "hydravms-super-secret-production-key-2026-auth-token-guard"  
}  
``
```

Impacto

Qualquer atacante que conheça o código aberto pode forjar tokens JWT arbitrários com qualquer UUID de tenant e papel de administrador.

Sugestão de Correção

Remover o fallback estático e emitir `log.Fatalf` se `JWT\_SECRET` estiver vazio ou possuir menos de 32 bytes.

Critérios de Aceite

```
- [ ] Servidor recusa iniciar sem `JWT_SECRET` válido.  
- [ ] Tokens forjados com a chave antiga são rejeitados.  
--- FIM ISSUE 3 ---
```

--- ISSUE 4 ---

[Segurança] Arbitrary File Read / Path Traversal em endpoint de streaming de gravações
Labels: security, critical, backend, lfi

Descrição do Problema

O endpoint de stream de vídeo de gravações (`GET /api/v1/cameras/{id}/recordings/stream`) executa `http.ServeFile(w, r, chosen.S3Key)` diretamente no valor gravado em `s3\_key` sem sanitizar se o caminho aponta para arquivos do sistema operacional fora do diretório de gravações.

Evidência

```
- **Arquivo:** `internal/adapters/primary/http/camera_handler.go:297-308`  
``go  
if chosen.S3Key == "" {  
    writeError(w, http.StatusNotFound, "Recording file path not found")  
    return  
}  
http.ServeFile(w, r, chosen.S3Key)  
``
```

Impacto

Leitura arbitrária de arquivos confidenciais do servidor (como `/etc/passwd`, chaves SSH, arquivos `.env` e bancos de dados).

Sugestão de Correção

1. Validar que `s3\_key` esteja dentro do caminho raiz permitido usando `filepath.Clean` e `strings.HasPrefix`.
2. Rejeitar caminhos absolutos fora de `storage/` ou servir via MinIO S3 Stream em vez de `http.ServeFile`.

Critérios de Aceite

```
- [ ] Tentativas de acessar caminhos fora de `storage/` retornam `403 Forbidden`.  
--- FIM ISSUE 4 ---
```

--- ISSUE 5 ---

[Segurança] IDOR na exclusão de Nós de Cluster e Pools de Storage
Labels: security, high, idor, backend

Descrição do Problema

Os endpoints `DELETE /api/v1/cluster/nodes/{id}` e `DELETE /api/v1/storage/pools/{id}` executam a remoção direta do objeto pelo ID sem verificar se o recurso pertence ao tenant do usuário autenticado.

Evidência

```
- **Arquivo:** `internal/adapters/primary/http/cluster_node_handler.go:305-320`  
``go  
idStr := strings.TrimPrefix(r.URL.Path, "/api/v1/cluster/nodes/")  
id, err := uuid.Parse(idStr)  
h.repo.Delete(r.Context(), id) // Sem validação de tenant  
``
```

Impacto

Inquilinos maliciosos podem deletar nós de GPU/Ingestão e pools de armazenamento de outros inquilinos ou do cluster principal.

Sugestão de Correção

Passar `tenantID` para o método `Delete(ctx, tenantID, id)` e incluir `WHERE id = \$1 AND tenant\_id = \$2` na query SQL.

Critérios de Aceite

```
- [ ] Tentativa de excluir nó ou pool de outro tenant retorna `404 Not Found` ou `403 Forbidden`.  
--- FIM ISSUE 5 ---
```